# Scientific writing made easy 101: Sublime and R-IDE

Last updated on 2018-10-10 · 8 min read

m/intent/tweet?text=Scientific%20writing%20made%20easy%20101%3a%20Sublime%20and%20R-
2fsublime_ride%2f)

book.com/sharer.php?u=%2fpost%2fsublime_ride%2f)

edin.com/shareArticle?
fpost%2fsublime_ride%2f&title=Scientific%20writing%20made%20easy%20101%3a%20Sublime%20and%20R-

ibo.com/share/share.php?
blime_ride%2f&title=Scientific%20writing%20made%20easy%20101%3a%20Sublime%20and%20R-IDE)

Scientific%20writing%20made%20easy%20101%3a%20Sublime%20and%20R-
t%2fsublime_ride%2f)

A couple of months ago, I was looking for an alternative to LaTex to easily write multi format articles and technical reports in a nicely formatted way. The natural choice would be the excellent R-Studio suite with RMarkdown and its new Radix framework. Although R-Studio is a good choice for small reports, it lacks most of the features of a modern writing suite, making the act of writing extremely annoying. I ran into Sublime thanks to this nice blog post (https://akosm.netlify.com/post/using-r-markdown-to-write-papers-that-look-seriously-nice/) by Akos Mate and I decided to give it a try. After more than two months of intensive use, I can undoubtedly say that Sublime is the clear winner among all its competitors (Latex, R-Studio, Atom, Vim).

## Save time and energy: the “One Text Editor - All the tasks” approach

Sublime is a free (but unfortunately closed source) multi-syntax text editor. Its strength resides in its easy and accessible Python architecture that allows users to write third-part plug-ins that enhance its basic functionalities and add new features. Thanks to its extreme versatility, its very active community developed a series of plug-ins that transform Sublime in a perfect multi-purpose programming and writing tool.

| | LaTex (basic) | R-Markdown (R-Studio) | Sublime |
|---|---|---|---|
| Multi syntax | No | Partially | Yes |
| Native multi format output | No | Yes | Yes |
| Easy integration with R or Python engines | No | Yes | Yes |
| Auto-complete citations | No | No | Yes |
| GitHub support | No | Partially | Yes |

| | | | |
|---|---|---|---|
| Project management features | No | Yes | Yes |
| Writing suite features (such as word count, word repetition…) | No | No | Yes |
| Distraction free mode | No | No | Yes |

Table: Some features comparison between LaTex, R-Studio, and Sublime

With some minimal tuning, you can have all the advantages of a modern text suit without losing the integration with R-Studio and Pandoc. In other words, you can use Sublime as your main working tool to program, update websites or repositories, and write scientific papers, small and large technical reports, blog posts, or even newspaper articles. Bear in mind that a basic knowledge of R, R-Markdown and shell console is required.

## What I need to start: 10 easy steps

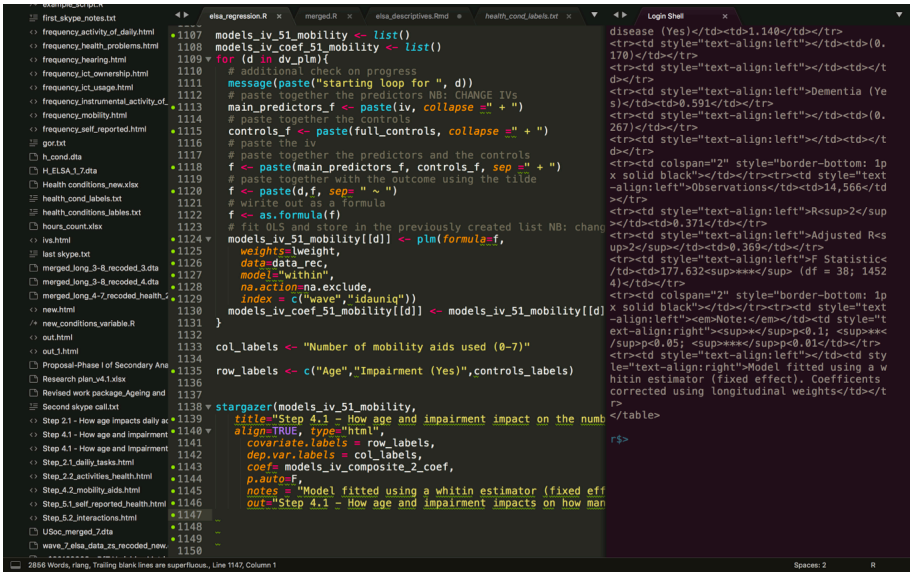
1. Download (https://www.sublimetext.com/) and install Sublime. If you are using MacOs you can install it trough HomeBrew (https://brew.sh/) using the command `brew cask install sublime-text` in your Terminal app.

2. Open Sublime and follow these instructions (https://packagecontrol.io/installation) to install its package manager **PackageControl**

3. Open the command palette in `Tools > Command Palette`. Select `Package Control: Advance Install Package` and copy and paste the following list of packages `R-IDE, LSP, Citer, SendCode, Terminus, BracketHighlighter, WordCount`. If the previous command fails for any reason (some package could change name or be eliminated), install the packages one by one manually using `Package Control: Install Package`

    - **R-IDE**: A package maintained by randy3k (https://github.com/randy3k) that transforms Sublime into R-Studio with some cool features added.
    - **LSP**: An implementation of Microsoft Language Server Protocol for various programming languages. It provides a syntax check and function description.
    - **Citer**: BibLatex citation handler with citation completions (when you hit @)
    - **SendCode**: Send your code to a terminal or program such as Terminus/Terminal/R-GUI/R-Studio.
    - **Terminus**: Run a terminal shell within Sublime Text.
    - **BracketHighlighter**: Advanced bracket highlighting. Extremely useful to spot unmatched parenthesis in your code.
    - **WordCount**: Count pages and words in your document.
4. Shutdown Sublime completely and re-open it.

5. Go `Tools > Layout > Columns: 2`. This command splits the Sublime main windows into two parts. Move the cursor on the right column, open the command palette `Tools > Command Palette` and select `Terminus: Open Default Shell in View`. This modular organisatoion of our workspace allows us to have the script or the main text on the left and the console (Terminus) on the right. **Note** You can further customize your workspace playing around with the `View` settings.




6. Install the R package `languageserver` (https://github.com/REditorSupport/languageserver) from CRAN. You can type `R` in the console that you just open in the right column of your Sublime Text. **NB**: R (and R-GUI) is different from R-Studio and they do not share the same package library. If you commonly use R-Studio and you switch to base R, remember to install the CRAN packages that you want to use.

```
# install R package languageserver
install.packages("languageserver")
```

7. Set up SendCode to use Terminus, `Tools > Command Palette > SendCode: Choose Program` and select `Terminus`. If you don't like to have a console in your working environment you can send your code to an external program such as R-GUI, R-Studio, or your default shell terminal. Even if it is not necessary, I strongly advise to install and use `rtichoke` as r terminal (see point 11 below).

8. Set up Citer. Go to `Sublime Text > Preferences > Package Settings > Citer` and paste the following code in setting widows that just opened. Change the bibtex_file_path. You can obtain a .bib file using your favourite citation manager (such as Zotero or Mendeley).

```
{
    //REQUIRED:

    "bibtex_file_path": "YOURPATH/YOURLIBRARY.bib",
    // You can also specify a list
    //"bibtex_file_path": ["example/path/to/file.bib", "example/path/to/fileTwo.bi
b"],

    //OPTIONAL:

    //By default Citer Search looks for your keyword in the
    //author, title, year, and Citekey (id) fields
    "search_fields": ["author", "title", "year", "id"] ,
    //Default format is @Citekey
    "citation_format": "@%s",
    //list of scopes. Could be top level "text" or "source", or limit to
    // e.g "text.html.markdown"
    "completions_scopes": ["text"],
    "enable_completions": true,
    //Customise the quickview of you library, using python format syntax
    "quickview_format": "{citekey} - {title}",
    "auto_merge_citations": ture
}
```

9. Enable the Language Server Protocol. Go to `Tools > Command Palette` and select `LPS: Enable Language Server`. You can choose to enable it globally or just for a single project.

10. Your setup is completed meaning that we are almost done. Let's see how to use Sublime/R-IDE to run some code or knit a PDF.

    1. If you have followed the instructions carefully you should have an empty document `untitled` on the left part of the screen and your Terminus console with R open on the right. Paste in the empty document the code here under (or open your own R script). Click on the bottom right corner and select R as a language syntax (if you opened your own .R script it should be already correctly set). You can now send the code to Terminus selecting `R-IDE > Send Code`. Alternatively, you can send code also using the keys combination `command+return` on MacOs or `control+return` on Window (same as R-Studio).**NB:** If you do not see the R-IDE menu or SendCode is not working properly (nothing appears in the terminal on the right when you send the code), check the previous steps and/or quit and re-open Sublime.

       ```
       # A comment: this is a sample script to try out Sublime and SendCode functional
       ities.
       y=c(12,15,28,17,18)
       x=c(22,39,50,25,18)
       mean(y)
       mean(x)
       plot(x,y)
       ```

    2. Open another Sublime instance and an R-Markdown file of your choice (.rmd). If you don't have one ready yet you can use this draft template (https://github.com/albertostefanelli/rmarkdown_templates) hosted on my GitHub. Now you can either knit (compile) the document in different formats changing the YALM meta data or try out any portions of the R code embedded in your .rmd file following step 5 and step 9. To kint the document go to `Tools > Build With` and select from the prompt R-Markdown or use the shortcut `command+b` on MacOs or `control+b` on Window to directly knit the file with the default compiler.

11. Fine tuning **(Optional)**

    1. Install rtichoke to get an improved console for R with multiline editing and rich syntax highlight. Follow the instructions on the rtichoke GitHub page

(https://github.com/randy3k/rtichoke). Remember to type in Terminus (or in your bash terminal) `rtichoke` instead of `r`.

2. Install MarkdownEditing (https://packagecontrol.io/packages/MarkdownEditing) or MarkdownComplements (https://packagecontrol.io/packages/MarkdownComplements) for better Markdown support such as lists and tables.
3. Install a Git plug-in from the Package Control (point 2) such as this one (https://github.com/kemayo/sublime-text-git).
4. Tune R-IDE for a better writing experience. Create a file called `R Markdown.sublime-settings` in `/Sublime Text 3/Packages/User` and paste the following code.

```
        // These settings override both User and Default settings for the R Mar

        "tab_size": 4,
        "translate_tabs_to_spaces": true,
        "trim_trailing_white_space_on_save": false,
        "auto_match_enabled": true,

        // Layout
        "draw_centered": true,
        "word_wrap": true,
        "wrap_width": 100,
        "rulers": [],

        // Line
        "line_numbers": false,
        "highlight_line": false,
        "line_padding_top": 2,
        "line_padding_bottom": 3,

        "gutter": false,
        "scroll_past_end": true
```

5. Add a shortcut to add an automatic R code block.

   - Create a file in `Sublime Text 3/Packages/R-IDE/support` called `ide-r-chunk.sublime-snippet` and paste the following code:

```
        <snippet>
            <content><![CDATA[
        ```{r $1}
        $2
        ```
        ]]></content>
            <scope>text.html.markdown.rmarkdown</scope>
            <description>Insert knitr Markdown chunk</description>
            <!-- Optional: Set a tabTrigger to define how to trigger the snippe
            <!-- <tabTrigger>hello</tabTrigger> -->
            <!-- Optional: Set a scope to limit where the snippet will trigger
            <!-- <scope>source.python</scope> -->
        </snippet>
```

   - Open `Sublime Text > Preferences > Key Bindings` and paste in the window that just opened the following code:

```
        [
              {
            "keys": ["super+alt+c"],
            "command": "insert_snippet", "args": {"name": "Packages/R-IDE/sup
port/ide-r-chunk.sublime-snippet"},
            "context": [
              { "operand": "text.html.markdown.rmarkdown", "operator": "equal
", "match_all": true, "key": "selector" }
            ]
          }
        ]
```

- Now you can use the `command+alt+c` key combination to insert an R code block in your R-Markdown file

## What now?

Your Sublime Text setup is ready! Take your time to understand how it works and to customize it according to your needs.

If you have any questions or suggestions about this guide, do not hesitate to contact me!

**Enjoy Sublime/R-IDE and happy writing.**

© 2018 · Powered by the Academic theme (https://sourcethemes.com/academic/) for Hugo (https://gohugo.io).